

Selection of k -th smallest element

(ADA-2023, CSE222)

Diptapriyo Majumdar

January 30, 2023

Deterministic Selection

Problem definition:

- **Input:** An array A of n distinct numbers and an integer $k \in \{1, \dots, n\}$.
- **Goal:** Find the k 'th smallest element of A . $\geq$

Some Questions:

- What can be some possible algorithms and their running times?

Sort the array. $O(n \log n)$ $k=1$
Then return $A[k]$.

$k=1$: initialize $\text{min} = A[1]$; $O(n)$?
for ($i=2$ to n) {
 if $A[i] < \text{min}$, then $\text{min} = A[i]$;
}

} return min;

Deterministic Selection

$(n-1)$
comparisons
and updates of min.

$O(kn)$

$O(n)$

Problem definition:

- **Input:** An array A of n distinct numbers and an integer $k \in \{1, \dots, n\}$.
- **Goal:** Find the k 'th smallest element of A .

Some Questions:

- What can be some possible algorithms and their running times?
- **Method:** Sort the array A , and find k 'th element in sorted array. It takes $O(n \log n)$ -time.

Input Instance

3	7	1	9	4	5	2	8	12	6	11	10
---	---	---	---	---	---	---	---	----	---	----	----

Input Instance

3	7	1	9	4	5	2	8	12	6	11	10
---	---	---	---	---	---	---	---	----	---	----	----

- If $k = 1$, then the problem is finding smallest element, that takes $\mathcal{O}(n)$ -time.

Input Instance

3	7	1	9	4	5	2	8	12	6	11	10
---	---	---	---	---	---	---	---	----	---	----	----

- If $k = 1$, then the problem is finding smallest element, that takes $\mathcal{O}(n)$ -time.
- There is no sublinear ($\mathcal{O}(n^{1-\varepsilon})$ -time for some $\varepsilon > 0$) algorithm that computes minimum element in an array.

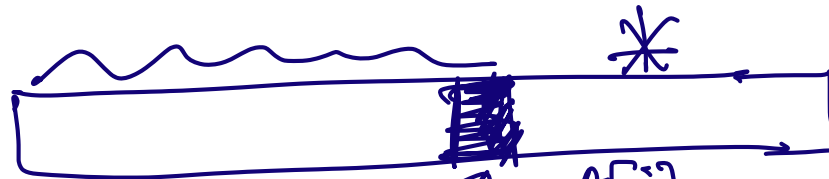
Can be proved by adversary argument.

Linear Time Selection

- Finding k 'th smallest element in an array can also be performed by an algorithm too. $O(n)$ -time
main agenda.

Algorithm (crucial ideas):

- Partition the input array A into two pieces.



if (pivot = $A[k]$)

→ then we are done (pivot)

if (pivot = $A[i]$ and $i > k$)

then Recurse on $A[1, 2, \dots, i]$

if pivot $A[i]$ and $i < k$.

$(n-i)$ elements are larger than
pivot $A[i, i+1, \underline{k}, \dots, n]$

k 'th smallest element is $(n-(k-i))$ th

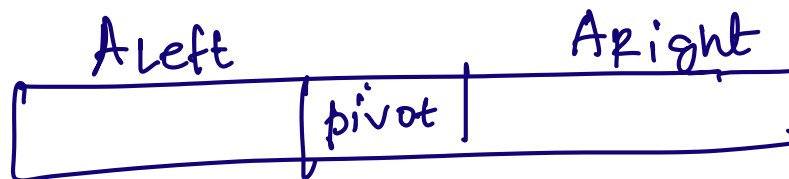
Algorithm (crucial ideas):

- Partition the input array A into two pieces.
- Choose an element p called **pivot**.
- A_{left} the subset of elements smaller than **pivot**.
- A_{right} the subset of elements larger than **pivot**.

Algorithm (crucial ideas):

- Partition the input array A into two pieces.
- Choose an element p called **pivot**. $\rightarrow$ How to choose?
- A_{left} the subset of elements smaller than **pivot**.
- A_{right} the subset of elements larger than **pivot**.
- The k 'th smallest element will be either in A_{left} or in A_{right} or the **pivot** itself.
- If $|A_{\text{left}}| = k - 1$, then **pivot** is the k 'th smallest.
- If $|A_{\text{left}}| > k - 1$, then k 'th smallest of A is the k 'th smallest of A_{left} .
- Otherwise, $|A_{\text{left}}| < k - 1$. The **pivot** is the $|A_{\text{left}}| + 1$ 'th smallest element of A . Then, k 'th smallest in A is the $(k - |A_{\text{left}}| - 1)$ 'th smallest in A_{right} .

$$r = |A_{\text{left}}|$$
$$r \leq k-2$$



$\rightarrow$ pivot is $(|A_{\text{left}}| + 1)$ 'th smallest from A .

Algorithm (crucial ideas):

- Partition the input array A into two pieces.
- Choose an element p called **pivot**. $\rightarrow$ How to choose?
- A_{left} the subset of elements smaller than **pivot**.
- A_{right} the subset of elements larger than **pivot**.
- The k 'th smallest element will be either in A_{left} or in A_{right} or the **pivot** itself.
- If $|A_{\text{left}}| = k - 1$, then **pivot** is the k 'th smallest.
- If $|A_{\text{left}}| > k - 1$, then k 'th smallest of A is the k 'th smallest of A_{left} . $\rightarrow$ Recursion
- Otherwise, $|A_{\text{left}}| < k - 1$. The **pivot** is the $|A_{\text{left}}| + 1$ 'th smallest element of A . Then, k 'th smallest in A is the $(k - |A_{\text{left}}| - 1)$ 'th smallest in A_{right} . $\rightarrow$ Recursion
- **Question:** How do we partition the array?

Recursion {

} Recursion

Algorithm

Select(A, n, k)

if $n = 1$ **then**

| Return $A[1]$;

end

$p \leftarrow$ ChoosePivot(A, n); $\rightarrow$ How we choose? $O(n)$ -time

$A_{\text{left}} \leftarrow \{A[i] : A[i] < p\}$; $A_{\text{right}} \leftarrow \{A[i] : A[i] > p\}$;

if $|A_{\text{left}}| = k - 1$ **then**

| Return p ;

end

if $|A_{\text{left}}| > k - 1$ **then**

| Return **Select**($A_{\text{left}}, |A_{\text{left}}|, k$);

end

else

| Return **Select**($A_{\text{right}}, |A_{\text{right}}|, k - |A_{\text{left}}| - 1$);

end

If pivot is $\frac{n}{2}$ 'th smallest.

We do not want
 $T(n) = T(n-1) + O(n)$

then $T(n) = T(\frac{n}{2}) + O(n)$

Possible Ideas

- How does the ChoosePivot work?
- The purpose is to find an element p , place p at its proper location in sorted array.
- Then, all elements smaller than p should be on the left of p , and all elements larger than p should be on the right of p .

Some possible ideas

- **Idea 1:** Choose a random pivot.

If pivot is always largest or smallest -
then recurrence (running time of algorithm)

is $T(n) = T(n-1) + O(n)$

$$\approx O(n^2).$$

Not useful

Some possible ideas

- **Idea 1:** Choose a random pivot.
What if this random pivot always chooses the *maximum element*?

Some possible ideas

- **Idea 1:** Choose a random pivot.
What if this random pivot always chooses the *maximum element*?
- **Idea 2:** Always choose median.

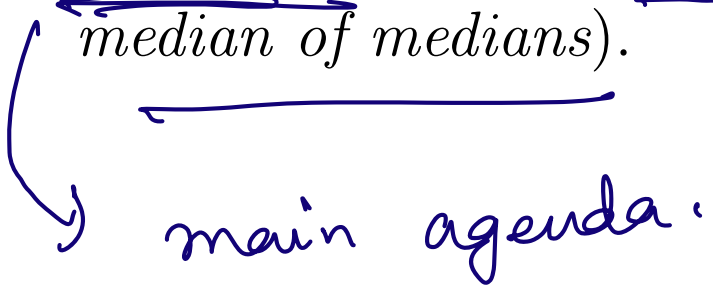
There is $O(n)$ -time algorithm.

But we do not know.

Some possible ideas

- **Idea 1:** Choose a random pivot.
What if this random pivot always chooses the *maximum element*?
- **Idea 2:** Always choose median.
But that requires to find median in $\mathcal{O}(n)$ -time. But we do not know yet.

Some possible ideas

- **Idea 1:** Choose a random pivot.
What if this random pivot always chooses the *maximum element*?
- **Idea 2:** Always choose median.
But that requires to find median in $\mathcal{O}(n)$ -time. But we do not know yet.
- **Idea 3:** Find a pivot *close enough* to the median (called median of medians).
 *main agenda.*

Using Idea 3

ChoosePivot(A, n)

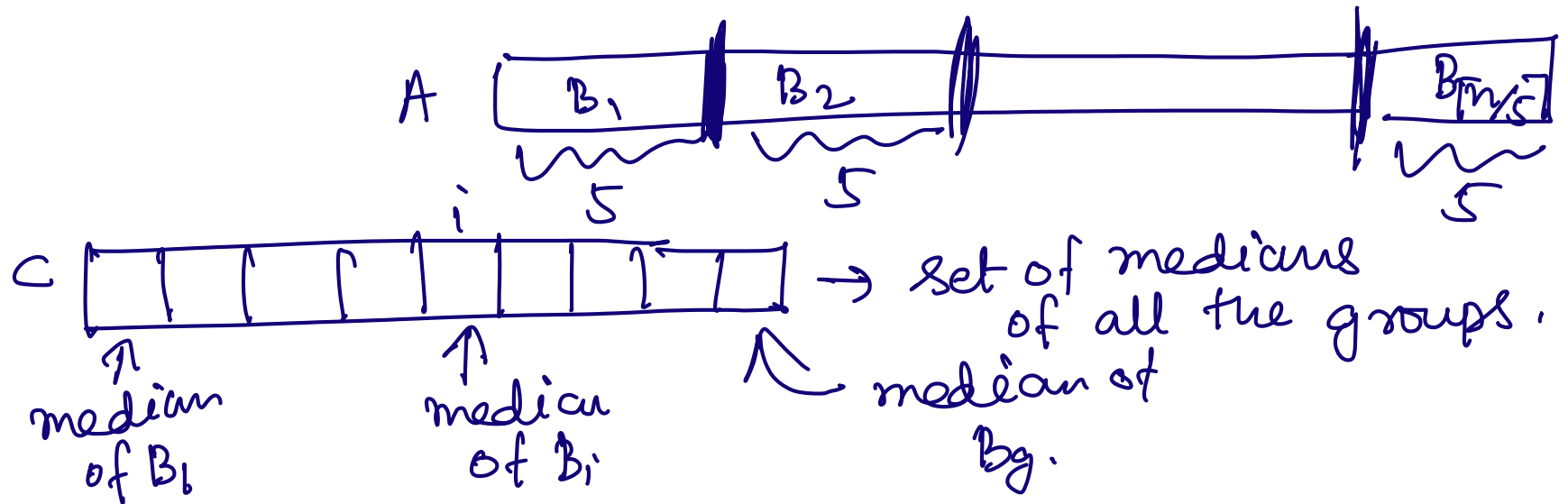
Split A into $g = \lceil n/5 \rceil$ groups $B_1, \dots, B_{\lceil n/5 \rceil}$ of size 5 each;

$\left\{ \begin{array}{l} \text{for } i = 1, \dots, g \text{ do} \\ \quad | \text{ Sort } B_i \text{ using Merge Sort;} \\ \text{end} \end{array} \right\} \rightarrow O\left(\frac{n}{5} 5 \log 5\right) \approx O(n)\text{-time}$

$C \leftarrow \{b_i : \text{median of } B_i, i = 1, \dots, g\};$

$p \leftarrow \text{Select}(C, g, g/2);$ $\rightarrow$ median of $C \rightarrow g/2$ 'th smallest element from C

Return p ;



Using Idea 3

ChoosePivot(A, n)

Split A into $g = \lceil n/5 \rceil$ groups $B_1, \dots, B_{\lceil n/5 \rceil}$ of size 5 each;

for $i = 1, \dots, g$ do

 | Sort B_i using Merge Sort;

end

$C \leftarrow \{b_i : \text{median of } B_i, i = 1, \dots, g\};$

$p \leftarrow \text{Select}(C, g, g/2);$

Return p ;

- Sorting an array of size 5 takes $\mathcal{O}(1)$ -time. Hence, sorting $\lceil n/5 \rceil$ distinct arrays will take $\mathcal{O}(n)$.
- Select subroutine is called on arrays of size $n/5$.

Overall algorithm

Select(A, n, k)

if $n = 1$ then

| Return $A[1]$; $\checkmark$

end

$p \leftarrow \text{ChoosePivot}(A, n)$; $\rightarrow O(n)$ -time

$\left\{ \begin{array}{l} A_{\text{left}} \leftarrow \{A[i] : A[i] < p\}; \\ A_{\text{right}} \leftarrow \{A[i] : A[i] > p\}; \end{array} \right\} \rightarrow O(n)$ -time

if $|A_{\text{left}}| = k - 1$ then

| Return p ;

end

if $|A_{\text{left}}| > k - 1$ then

| Return Select($A_{\text{left}}, |A_{\text{left}}|, k$);

end

else

| Return Select($A_{\text{right}}, |A_{\text{right}}|, k - |A_{\text{left}}| - 1$);

end

ChoosePivot(A, n)

Split A into groups of

size $g = \lceil n/5 \rceil$ each

$B_1, \dots, B_{\lceil n/5 \rceil}$;

for $i = 1, \dots, g$ do

| Sort B_i using

| Merge Sort;

end

$C \leftarrow \{b_i : \text{median of}$

$B_i, i = 1, \dots, g\}$;

$p \leftarrow \text{Select}(C, g, g/2)$; $T(n/5)$

Return p ;

Induction on $|A| = n$.

Correctness Proof

Base Case: $n=1$. Trivial.

Induction Hypothesis: The algorithm outputs correct answer
for all $n=1, 2, \dots, n-1$.

Induction Step: $n=n$.

Choose pivot sorts groups of 5 elements.
then calls recursive subroutine on an array of
 $n/5$ elements.

By induction hypothesis: median of medians is
correctly identified. This is pivot.

If $|A_{\text{left}}| = k-1 \Rightarrow$ pivot is k^{th} smallest
 $\Rightarrow$ output is correct

If $|A_{\text{left}}| > k-1 \Rightarrow$ k^{th} smallest of A is
 k^{th} smallest element of A_{left} .
 $n < \infty$
By induction hypothesis.

If $|A_{\text{left}}| < k-1 \Rightarrow$ k^{th} smallest element of A
is $(k - |A_{\text{left}}| - 1)^{\text{th}}$ smallest
element of A_{right} .
 $n < \infty$
By induction hypothesis.

Analysis of running time

- What is $|A_{\text{left}}|$ and $|A_{\text{right}}|$?

Lemma: $|A_{\text{left}}| \leq \frac{7n}{10} + 2$

Proof: pivot

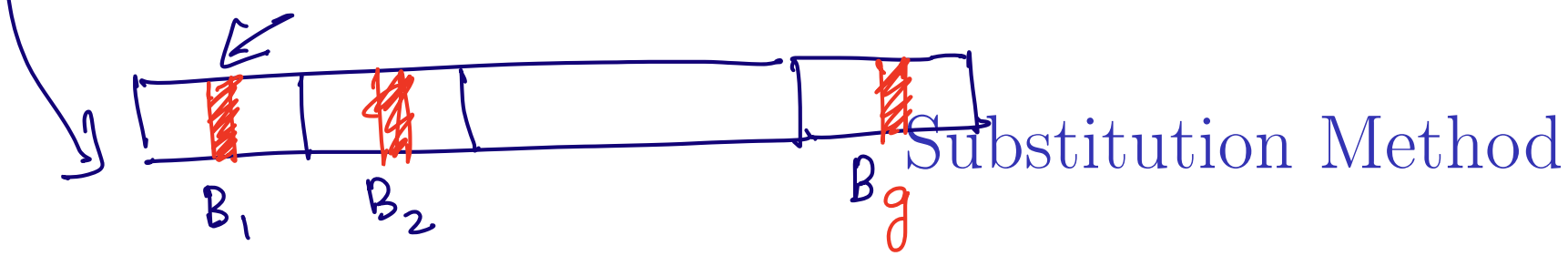


There are $\frac{n}{5}$ groups.

pivot is larger than at least 2 elements
from B_i .

$$g = \frac{n}{5}$$

Formally, for every $i=1, 2, \dots, g$
pivot is larger than at least 2 elements
from B_i .



pivot is larger than $\left(\frac{q}{2} - 1\right)$ elements of A.

These $\left(\frac{q}{2} - 1\right)$ elements belong to $\left(\frac{q}{2} - 1\right)$ distinct groups

Hence, pivot is larger than 2 additional elements from each of these $\left(\frac{q}{2} - 1\right)$ distinct groups.

Hence, pivot is larger Substitution Method

than $2\left(\frac{q}{2} - 1\right) + \left(\frac{q}{2} - 1\right)$ elements of A.

$$\boxed{q = \frac{n}{5}} = 2\left(\frac{n}{10} - 1\right) + \frac{n}{10} - 1 = \frac{3n}{10} - 3$$

pivot is larger than $\left(\frac{3n}{10} - 3\right)$ elements of A.


$$\begin{aligned} |A_{\text{left}}| &\leq (n-1) - \left(\frac{3n}{10} - 3\right) \\ &\leq \frac{7n}{10} + 2 \end{aligned}$$

$$|A_{left}| \leq \frac{7n}{10}$$

Substitution method

By symmetric, $|A_{right}| \leq \frac{7n}{10}$.

$$T(n) = T\left(\frac{7n}{10}\right) + T\left(\frac{n}{5}\right) + cn \quad \text{if } n > 1$$

$$= 1 \quad \text{if } n = 1$$

↓
use substitution method.

Reading material

- Will be provided in the google-classroom.

Jeff Erickson lecture note Section 1.8
(recursion.pdf).